

CPU vs GPU & Microprocessor — Detailed Lecture Notes

JKSSB Computer Awareness

October 7, 2026

Video lesson: [CPU vs GPU: What's the Difference? Microprocessor & Microcontroller Explained](#)

06.1 Serial and parallel processing

Serial processing means that one task is carried out at a time, in sequence: the first operation finishes before the next begins. **Parallel processing** means that several tasks or operations proceed at the same time. A serial workload rewards a processor with a few very fast units; a parallel workload rewards a processor with many units that can work simultaneously.

Definition (Serial processing). *Carrying out one task at a time, in sequence, so that each step completes before the next starts.*

Definition (Parallel processing). *Carrying out several tasks or operations at the same time.*

The difference matters because it explains why two processors can both be described as “fast” while being designed for opposite jobs. A processor that excels at finishing a single chain of instructions quickly is a latency-optimised design. A processor that excels at completing a very large number of small independent operations per second is a throughput-optimised design.

06.2 CPU versus GPU

The CPU (Central Processing Unit) is the general-purpose processor of a computer. It has **few, powerful cores**, and each core is capable of running a sequential instruction stream quickly. Its design goal is **low latency**: to complete one task with the least possible delay. In the office-desktop running example (EX-01), the CPU executes the operating system, MS Word, and spreadsheet logic step by step.

The GPU (Graphics Processing Unit) is a processor built around **many simpler cores**. Each core does a small amount of work, and the design goal is **high throughput**: a very large number of independent operations per second. This makes the GPU a natural fit for pixels and graphics, and also for scientific and machine-learning workloads that can be expressed as many parallel calculations.

Feature	CPU	GPU
Cores	Few, powerful	Many, simple
Design goal	Low latency	High throughput
Best suited to	Sequential, general-purpose work	Parallel work (pixels, ML)
Core type	General-purpose	Specialised, simpler
Example use	EX-01 desktop logic	Image and graphics operations

Example 06.2.1. *An item asks which processor is better suited to an image-processing task that repeats the same operation on millions of pixels. Because those pixel operations are independent and can run in parallel, the GPU — with its many simple cores — is the better fit. The CPU’s few powerful cores would each handle the chain of pixels one after another.*

The most common near-neighbour error is to treat GPU cores as if they were interchangeable general-purpose CPU cores. They are not. GPU cores are simpler and optimised for parallel throughput, so they are not a full replacement for the CPU’s general-purpose cores.

06.3 Cores, threads, and hyper-threading

A **core** is a physical processing unit. A quad-core CPU contains four such units. A **thread** is a stream of instructions that a core executes. **Hyper-threading** lets a single physical core present two logical threads, so the core can stay busy while one thread is waiting.

Two logical threads on one core are not two physical cores. A quad-core processor with hyper-threading may report eight logical threads, but it still has four physical cores. Clock speed and core count together, not thread count alone, determine raw capacity.

Definition (Hyper-threading). *A technique in which one physical core presents two logical threads to the operating system, improving utilisation while a thread waits.*

06.4 Cache depth: L1, L2, and L3

Cache is small, fast memory held close to the core. It is organised in levels. **L1** is the smallest and fastest level and sits nearest the core. **L2** is larger and a little slower than L1. **L3** is the largest and slowest of the three and is typically shared across cores. The order of speed and closeness is L1, then L2, then L3, then main memory.

Level	Relative size	Relative speed and location
L1	Smallest	Fastest; closest to the core
L2	Larger	Slower than L1
L3	Largest	Slowest of the three; often shared across cores

06.5 Clock speed and multi-core

Clock speed is measured in **GHz** (gigahertz), which counts billions of cycles per second. A higher clock speed generally means more work completed per second, all else being equal. A **multi-core** processor places several cores on one chip, which enables parallel work. However, clock speed alone never decides real performance: the number of cores, the cache size, the amount of memory, and the input/output speed all contribute.

06.6 Microprocessor versus microcontroller

A **microprocessor** is a central processing unit implemented on a single chip. It is a general-purpose processor, and software decides what it does. On its own it still needs external memory

and input/output devices to form a complete working system. The EX-01 office desktop is built around a microprocessor.

A **microcontroller** integrates more on the same chip: a microprocessor **plus** memory **plus** input/output peripherals. It therefore forms a near-complete computer on one chip and is used for **embedded/special-purpose** control, for example in washing machines, remote controls, and appliance or vehicle controllers.

Definition (Microprocessor). *A central processing unit implemented on a single chip; a general-purpose device that needs external memory and I/O.*

Definition (Microcontroller). *A microprocessor together with integrated memory and input/output peripherals on one chip, used in embedded and special-purpose systems.*

Feature	Microprocessor	Microcontroller
Contents	CPU on a chip	CPU + memory + I/O on a chip
Purpose	General-purpose	Embedded / special-purpose
System	Needs external parts	Self-contained on one chip
Example	EX-01 office desktop	Appliance controller

The key difference is the **level of integration**, not clock speed. A microcontroller is not “just a microprocessor”: it is a more integrated device that already includes memory and I/O on the same chip.

06.7 Diagnosing a “slow” computer (EX-01)

The office desktop in this running example is a quad-core 2.4 GHz machine with 16 GB of RAM and an SSD. Suppose one particular program still runs slowly. A common but incorrect conclusion is that the CPU must be slow, because “more cores should mean faster”. More cores do not help a single-threaded program, which uses only one core at a time. The real cause is usually a **bottleneck**: memory pressure, disk input/output, or software that is not multithreaded.

Example 06.7.1. *A question describes a quad-core 2.4 GHz desktop with 16 GB RAM and an SSD, and asks why one application is slow. The best answer is to diagnose a stated bottleneck — a single-threaded program, insufficient memory, or slow disk input/output — rather than to declare the CPU slow. A slow program is not automatically a “slow CPU.”*

06.8 Exam retrieval and common confusions

Use two retrieval directions. Given the processor, recall its core profile; given the workload, recall the better processor.

- CPU $\leftrightarrow$ few powerful general-purpose cores, low latency.
- GPU $\leftrightarrow$ many simple cores, high throughput, parallel.
- Serial $\leftrightarrow$ one task at a time in sequence.
- Parallel $\leftrightarrow$ several tasks or operations at once.

- Microprocessor $\leftrightarrow$ CPU on a single chip, general-purpose.
- Microcontroller $\leftrightarrow$ microprocessor + memory + I/O on one chip, embedded.
- Cache order $\leftrightarrow$ L1 (smallest, fastest) to L3 (largest, slowest).

Three distinctions prevent most errors in this topic:

1. Do not treat GPU cores as general-purpose CPU cores. They are simpler and optimised for parallel throughput.
2. Do not confuse serial with parallel. Serial is sequential, one task at a time; parallel is simultaneous.
3. Do not attribute a slow program on a capable machine automatically to a slow CPU. Check the multithreading, memory, and disk-I/O bottleneck; and do not confuse a microprocessor (CPU on a chip) with a microcontroller (CPU + memory + I/O on one chip).

Exercises

1. Distinguish serial processing from parallel processing.
2. State one design goal of the CPU and one of the GPU, and give the core profile of each.
3. Explain the difference between a microprocessor and a microcontroller.
4. What does hyper-threading allow, and why is it not the same as adding a physical core?
5. A quad-core 2.4 GHz desktop with 16 GB RAM and an SSD runs one program slowly. Give the most appropriate diagnosis and say why.

Solutions

1. Serial processing performs one task at a time in sequence; parallel processing performs several tasks or operations at the same time.
2. The CPU is optimised for low latency and has few powerful general-purpose cores. The GPU is optimised for high throughput and has many simpler cores for parallel work.
3. A microprocessor is a CPU on a single chip and is general-purpose, needing external memory and I/O. A microcontroller adds memory and I/O peripherals on the same chip, forming an embedded, special-purpose system; the difference is the level of integration.
4. Hyper-threading lets one physical core present two logical threads, so it can stay busy while one thread waits. It does not add a physical core; a quad-core chip with hyper-threading still has four physical cores.
5. The program is likely limited by a bottleneck — a single-threaded workload, insufficient memory, or slow disk input/output — rather than by a slow CPU, since extra cores do not speed up a single-threaded program. A slow program is not automatically a “slow CPU.”